

ACHME Communication CRM — Full Project Error & Bug Report

Analyzed by reading every source file in the zip. Errors are grouped by severity.

CRITICAL — Will CRASH or BREAK on startup

1. Empty Route Files — Backend Will Crash on Require

Files affected:

File	Size	Problem
backend/routes/invoiceRoutes.js	1 byte	Effectively empty — <code>require()</code> returns <code>{}</code> , not a Router
backend/routes/sendotp.js	0 bytes	Completely empty — <code>require()</code> will throw
backend/routes/chatroutes.js	~2KB	Entire file is 100% commented out — no export

Where they are required in `server.js`:

```
js
// server.js line ~
app.use("/api/estimate-invoice", require("./routes/estimateInvoiceRoutes")); // ✓
```

`invoiceRoutes.js` and `sendotp.js` are NOT currently mounted in `server.js`, but they exist as files and will cause issues if anyone tries to `require` them. `chatroutes.js` is also not mounted — meaning the entire **Chat feature has no backend**.

Fix:

- Either implement these route files properly, or delete them.
- For `chatroutes.js`, uncomment the code and register the route in `server.js`:

```
js
app.use("/api/chat", require("./routes/chatroutes"));
```

2. All `chatbox/` Frontend Components Are Empty (0 Bytes)

Files:

```
frontend/src/chatbox/chatsidebar.jsx    - 0 bytes
frontend/src/chatbox/chatwindow.jsx     - 0 bytes
frontend/src/chatbox/contextpanel.jsx   - 0 bytes
frontend/src/chatbox/messagecompos.jsx  - 0 bytes
frontend/src/chatbox/messagesidebar.jsx - 0 bytes
```

Impact: If ANY component tries to import from these files, the React build will fail. The chat feature is completely non-functional — frontend AND backend are both empty.

Fix: Implement these components or remove the folder until the feature is ready.

3. `UnifiedInvoiceForm.jsx` Is Empty (0 Bytes)

File: `frontend/src/components/UnifiedInvoiceForm.jsx` — 0 bytes

App.js does NOT import this file, but it is referenced in other pages. Any page that tries to import it will crash the build.

Fix: Implement the component or remove the import references.

4. Duplicate Entry Point — `index.js` vs `main.jsx`

Files:

- `frontend/src/index.js` — CRA's real entry point (used by `react-scripts`)
- `frontend/src/main.jsx` — dead file, never executed by CRA

`main.jsx` contains TWO critical bugs even though it's not used:

Bug A — Wrong import path:

```
jsx
```

```
// main.jsx - WRONG:
import { AuthProvider } from "../auth/AuthContext"; // ❌ goes up one folder - file

// Should be:
import { AuthProvider } from "./auth/AuthContext"; // ✅
```

Bug B — Double AuthProvider wrap: `main.jsx` wraps `<App />` in `<AuthProvider>`, but `App.js` already renders `<AuthProvider>` internally. This would cause duplicate context providers.

Fix: Delete `main.jsx` entirely. It is a leftover from a Vite project template. CRA uses only `index.js`.

5. React 19 + `react-scripts` 5.0.1 Incompatibility

`frontend/package.json`:

```
json
"react": "^19.2.3",           ← React 19
"react-scripts": "5.0.1"     ← Was built for React 17/18
```

`react-scripts` 5.0.1 was released in 2022 and is not tested or supported with React 19. This causes:

- Build warnings or failures in TypeScript type checks
- Potential issues with `@testing-library/react` version conflicts
- `npm install` peer dependency warnings/errors

Fix options:

- Downgrade React to `^18.3.1` (recommended for `react-scripts` 5), OR
 - Migrate the project to **Vite** (modern, actively maintained)
-

● HIGH — Features Are Broken

6. Chat Feature Is Completely Non-Functional

Three-layer failure:

Layer	Problem
Backend route	<code>chatroutes.js</code> — 100% commented out, not mounted in <code>server.js</code>
Frontend components	All 5 <code>chatbox/</code> files are 0 bytes
Socket registration	Chat socket IS initialized in <code>server.js</code> via <code>chatsockets.js</code> , but has no REST API counterpart

The Socket.IO chat socket in `backend/sockets/chatsockets.js` works (it's not empty), but without a REST API for fetching chat history and the frontend components, chat is completely broken.

7. `adminnotifications.jsx` Page Has No Backend Route

File: `frontend/src/pages/adminnotifications.jsx` (35KB — large, fully implemented page)

Looking at `server.js`, the notifications routes registered are:

```
js
app.use("/api/notifications", require("../routes/notificationRoutes"));
```

The `adminnotifications.jsx` page likely calls different endpoints. There is no `adminNotificationRoutes.js` file in `backend/routes/`. This means admin notification fetches will either 404 or silently fail.

Fix: Audit what API endpoints `adminnotifications.jsx` calls and ensure they exist in `notificationRoutes.js`.

8. `socket.io` Listed as Frontend Dependency (Should Be `socket.io-client` Only)

frontend/package.json:

```
json
```

```
"socket.io": "^4.8.3",           ← Server-side package – should NOT be here
"socket.io-client": "^4.8.3"     ← ✓ This is the correct one for browsers
```

Having `socket.io` (the server package) in the frontend adds ~5MB of unnecessary weight to the bundle and can cause unexpected behavior.

Fix: Remove `socket.io` from `frontend/package.json`. Keep only `socket.io-client`.

● MEDIUM — Bugs That Cause Incorrect Behavior

9. Five Duplicate / Conflicting API Configuration Files

There are **5 separate files** that define the API base URL, and they don't agree:

File	What it returns in production
<code>src/config.js</code>	<code>REACT_APP_API_URL</code> env var OR dynamic <code>window.location</code>
<code>src/config/index.js</code>	<code>""</code> (empty string) in production
<code>src/config/api.js</code>	Imports from <code>config/index.js</code> → <code>""</code> in production
<code>src/config/axios.js</code>	Uses <code>config/index.js</code> → <code>""</code> in production
<code>src/api/axios.js</code>	Hardcoded <code>/api</code>
<code>src/api/base.js</code>	<code>/api</code> in production, dynamic in dev
<code>src/App.js</code>	Defines its OWN inline <code>getApiBackend()</code> function

The problem: `src/config/index.js` returns `""` in production:

```
js
// config/index.js
const getApiUrl = () => {
  if (process.env.NODE_ENV === "production") return ""; // ← Returns EMPTY STRING
  ...
};
```

This means ALL components importing from `config/index.js` → `config/api.js` will use relative URLs in production (which only works if Nginx proxies `/api` — but that's not configured anywhere in this project yet).

Meanwhile `App.js` and `src/config.js` use `REACT_APP_API_URL` env var, which IS correct.

Fix: Pick ONE approach and use it everywhere. The recommended approach for LAN deployment:

```
js

// Single config/index.js for all files:
export const API = process.env.REACT_APP_API_URL ||
  `${window.location.protocol}://${window.location.hostname}:5000`;
```

10. `setupProxy.js` Hardcoded to `localhost` — Breaks LAN Dev Testing

```
js

// frontend/src/setupProxy.js
target: "http://localhost:5000", // ← Hardcoded!
```

When a developer runs the frontend from a laptop and accesses it from another machine on the LAN, the proxy still tries `localhost:5000` on the **server machine** — but during development from the dev machine, this is fine. However, if anyone wants to test from another LAN device during development (using the dev server), it won't work.

Fix for dev:

```
js

target: process.env.REACT_APP_BACKEND_URL || "http://localhost:5000",
```

11. MySQL Single Connection Instead of Pool — Will Drop Under Load

`backend/config/database.js` line 11:

```
js

const db = mysql.createConnection({ ... }); // ← Single persistent connection
```

A single connection has no reconnection logic, drops silently when MySQL's `wait_timeout` expires (default 8 hours), and cannot handle concurrent queries in parallel.

Fix: Use a connection pool:

```
js

const db = mysql.createPool({
  host: process.env.DB_HOST,
  port: dbPort,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: dbName,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  charset: "utf8mb4",
});
```

Then use `db.promise()` for async queries.

12. `multipleStatements: true` in DB Connection — SQL Injection Risk

`backend/config/database.js`:

```
js

const db = mysql.createConnection({
  ...
  multipleStatements: true // ← DANGEROUS
});
```

This allows an attacker to chain SQL statements. If any query parameter is unsanitized, this can lead to full database compromise.

Fix: Remove `multipleStatements: true` unless absolutely required for a specific migration script. Move those scripts to a separate one-time connection.

13. `backend/package.json` Has Wrong `main` Field

```
json
```

```
{
  "main": "index.js",    ← File does not exist
  "scripts": {
    "start": "node server.js" ← Actual entry point
  }
}
```

While this doesn't crash anything at runtime, it's misleading and will confuse tools that read `package.json`.

Fix:

```
json
{
  "main": "server.js"
}
```

● LOW — Security & Quality Issues

14. Real Credentials Committed to Git (`.env.example` Files)

Both `.env.example` (root) and `backend/.env.example` contain **real production secrets**:

```
EMAIL_PASS=ghjv omqm hwji kerq    ← Real Gmail App Password
JWT_SECRET=97418d0c15d57ade...     ← Real JWT secret in use
MYSQL_ROOT_PASSWORD=admin@123      ← Real DB password
```

These are committed to the public GitHub repo at https://github.com/Ananth-madura/ACHME_COMMUNICATION.git.

Immediate actions required:

1. Revoke and regenerate the Gmail App Password at <https://myaccount.google.com/apppasswords>
 2. Generate a new JWT secret: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`
 3. Change the MySQL root password
 4. Replace `.env.example` values with placeholder text like `your-gmail-app-password-here`
-

15. Large Binary Files Committed to Git Repository

File	Size
backend/test2.pdf	~910 KB
backend/test_output.pdf	~915 KB
frontend/src/images/achme logo high.jpg.jpeg	~678 KB
frontend/src/images/achme-logo-high.jpeg	~678 KB (duplicate!)
frontend/src/images/backhead.png	~777 KB

Total unnecessary binary weight: ~4 MB in git. These bloat `git clone` time and CI/CD pipelines.

Fix: Add to `.gitignore` and use a CDN or static asset server for production images.

16. `frontend/src/images/` Has Duplicate Files

```
achme logo high.jpg.jpeg    ← Has space in name (will cause import issues on Linux)
achme-logo-high.jpeg        ← Same file, hyphen version (678KB each – duplicated!)
```

Filenames with spaces cause import errors on Linux/macOS:

```
js
import logo from './images/achme logo high.jpg.jpeg'; // ← Will fail on Linux
```

Fix: Use only `achme-logo-high.jpeg` (no spaces), delete the space version.

17. Puppeteer Listed as Backend Dependency

```
json
"puppeteer": "^24.40.0" // ~150MB download + Chromium
```

Puppeteer downloads a full Chromium browser (~150-300MB) during `npm install`. On a production server this:

Fix: If only used for PDF generation, consider using `puppeteer-core` with a system-installed Chrome, or switch to a lighter alternative like `pdfkit` or `@react-pdf/renderer`.

18. `http-proxy-middleware` Missing from `frontend/package.json` Dependencies

`frontend/src/setupProxy.js` uses `http-proxy-middleware`, but it's not listed in `frontend/package.json`. It works currently because it's a transitive dependency of `react-scripts`, but this is fragile.

Fix:

```
bash
cd frontend && npm install --save-dev http-proxy-middleware
```

19. No PostCSS Config for Tailwind CSS

`tailwind.config.js` exists but there is no `postcss.config.js` in the frontend directory. Tailwind CSS v3 requires PostCSS to process the directives in `tailwind.css`.

`react-scripts` 5 has built-in PostCSS support and will pick up `tailwind.config.js` automatically if `tailwindcss` is in `devDependencies` — so this may work in practice, but it's relying on implicit behavior.

Fix: Add `postcss.config.js` explicitly:

```
js
// frontend/postcss.config.js
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

Summary Table

#	Severity	Issue	File(s)
1	 Critical	Empty/commented backend route files	<code>routes/invoiceRoutes.js</code> , <code>sendotp.js</code> , <code>chatroutes.js</code>
2	 Critical	All chatbox components are 0 bytes	<code>src/chatbox/*.jsx</code>
3	 Critical	UnifiedInvoiceForm is empty	<code>components/UnifiedInvoiceForm.jsx</code>
4	 Critical	Duplicate entry point + wrong import path	<code>src/main.jsx</code>
5	 Critical	React 19 + react-scripts 5 incompatibility	<code>frontend/package.json</code>
6	 High	Chat feature broken end-to-end	Both frontend & backend
7	 High	Admin notifications has no backend route	<code>adminnotifications.jsx</code>
8	 High	<code>socket.io</code> server pkg in frontend	<code>frontend/package.json</code>
9	 Medium	5 conflicting API config files	<code>src/config/</code> , <code>src/api/</code>
10	 Medium	Proxy hardcoded to localhost	<code>setupProxy.js</code>
11	 Medium	Single DB connection, no pool	<code>config/database.js</code>
12	 Medium	<code>multipleStatements: true</code> SQL risk	<code>config/database.js</code>
13	 Medium	Wrong <code>main</code> in package.json	<code>backend/package.json</code>
14	 Low	Real secrets in <code>.env.example</code> (in git!)	<code>.env.example</code>

#	Severity	Issue	File(s)
15	 Low	Large binary files in git	<code>backend/</code> , <code>frontend/src/images/</code>
16	 Low	Duplicate images + spaces in filename	<code>src/images/</code>
17	 Low	Puppeteer is too heavy for deployment	<code>backend/package.json</code>
18	 Low	<code>http-proxy-middleware</code> not in deps	<code>frontend/package.json</code>
19	 Low	No <code>postcss.config.js</code>	<code>frontend/</code> root